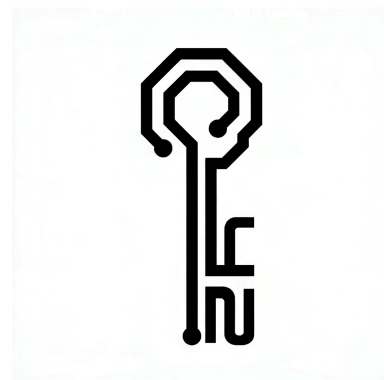


Национальный исследовательский ядерный университет «МИФИ»

Институт интеллектуальных кибернетических систем

Кафедра № 42 «Криптология и кибербезопасность»



ОТЧЕТ

**О выполнении лабораторной работы № 6
«Коллективные операции в MPI»**

Студент: Доценко В. А.

Группа: Б23-505

Преподаватель: Куприяшин М.А.

Москва — 2025

Оглавление

1. Цель работы	3
2. Рабочая среда	3
3. Описание MPI-алгоритма	4
3.1. Блок-схема алгоритма	4
3.2. Описание использованных функций	5
3.2.1. MPI_Init()	5
3.2.2. MPI_Comm_rank()	5
3.2.3. MPI_Comm_size()	5
3.2.4. MPI_Scatterv()	5
3.2.5. MPI_Gatherv()	5
3.2.6. MPI_Wtime()	5
3.2.7. MPI_Finalize()	5
4. Результаты экспериментов	6
4.1. Дополнительное задание	7
5. Заключение	10
6. Приложение. Разработанные программные коды	12

1. Цель работы

Приобрести навыки применения коллективных операций при разработке параллельных программ на основе технологии MPI

2. Рабочая среда

Модель процессора: Apple M3.

Количество ядер / потоков: 8 / 8.

Объем оперативной памяти: 24 ГБ.

Разрядность и версия ОС: macOS Sequoia 15.6.

Среда разработки: VS Code, clang 16.0.0.

Версия OpenMP: 4.0 (`_OPENMP = 202011`).

3. Описание MPI-алгоритма

3.1. Блок-схема алгоритма

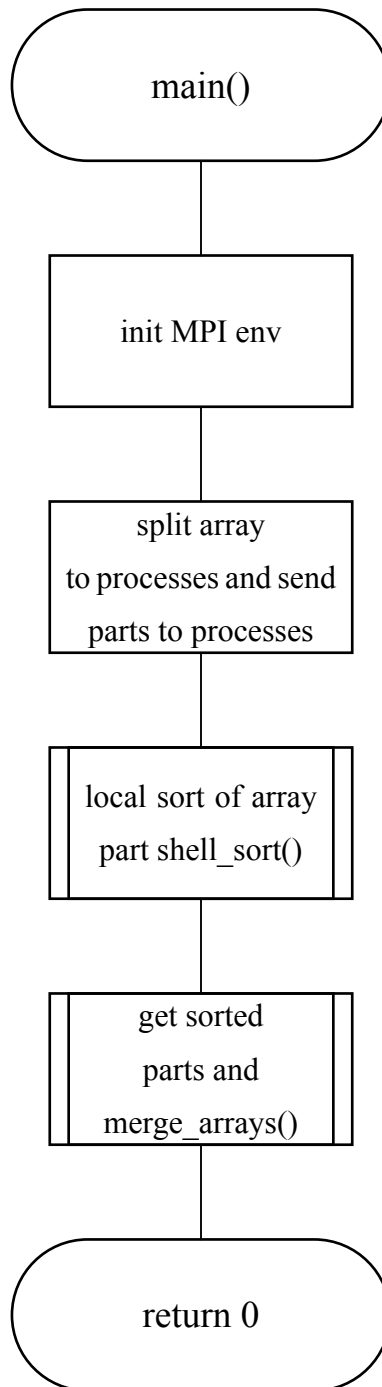


Рисунок 1 – Блок-схема алгоритма сортировки Шелла с использованием MPI

3.2. Описание использованных функций

3.2.1. MPI_Init()

Запускает и настраивает среду MPI, подготавливая процессы к обмену сообщениями.

3.2.2. MPI_Comm_rank()

Определяет номер (идентификатор) текущего процесса внутри коммуникатора.

3.2.3. MPI_Comm_size()

Сообщает, сколько процессов входит в коммуникатор.

3.2.4. MPI_Scatterv()

Отправка данных с переменным размером блоков — передаёт от корневого процесса к каждому процессу коммуникатора части массива, размеры которых могут различаться.

3.2.5. MPI_Gatherv()

Получает от всех процессов массивы различного размера и складывает их в общий буфер на корневом процессе.

3.2.6. MPI_Wtime()

Возвращает текущее время в секундах с высокой точностью.

3.2.7. MPI_Finalize()

Завершает работу MPI-программы, освобождая ресурсы и корректно завершая все операции обмена.

4. Результаты экспериментов

Каждая точка на графике была получена путем повторения на 20 сгенерированных массивах размерами 1 млн. элементов. Массивы были одинаковы для всех точек графика. Распределение случайных элементов: от 0 до 1млн.

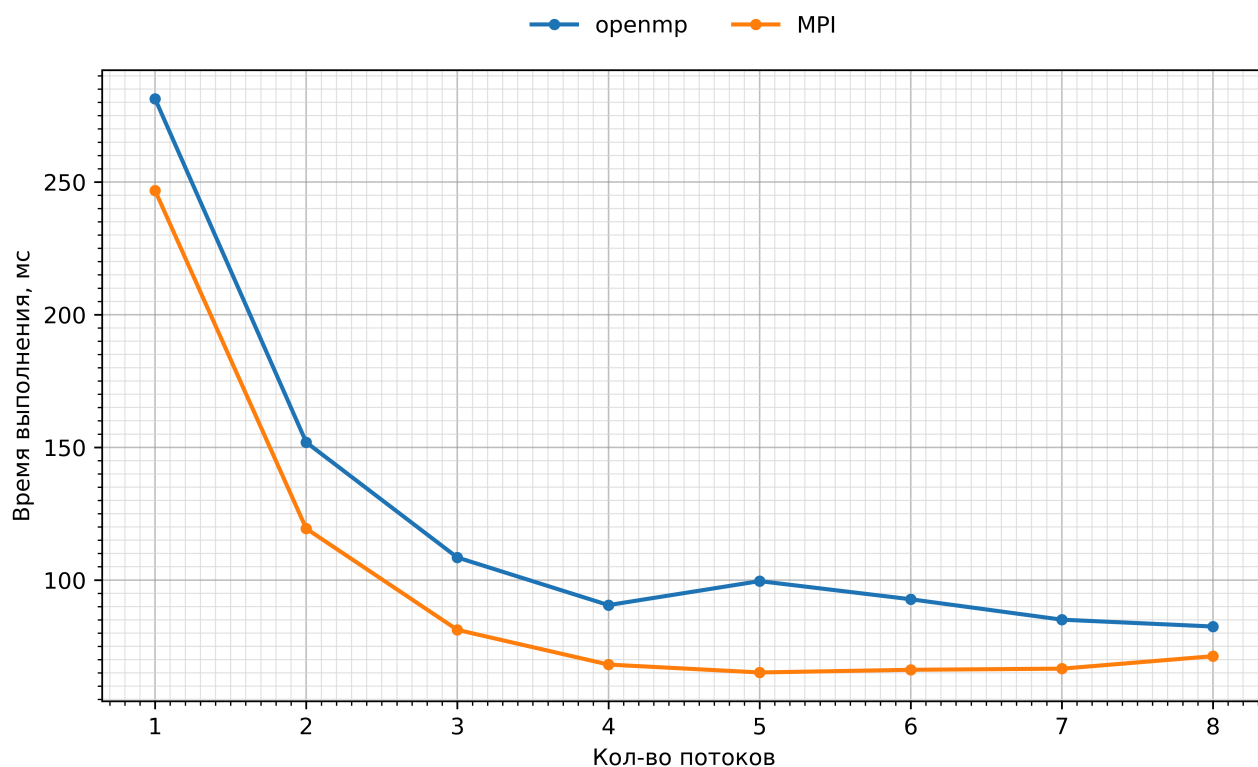


Рисунок 2 – График зависимости времени выполнения программы от количества потоков для MPI и OPENMP реализаций

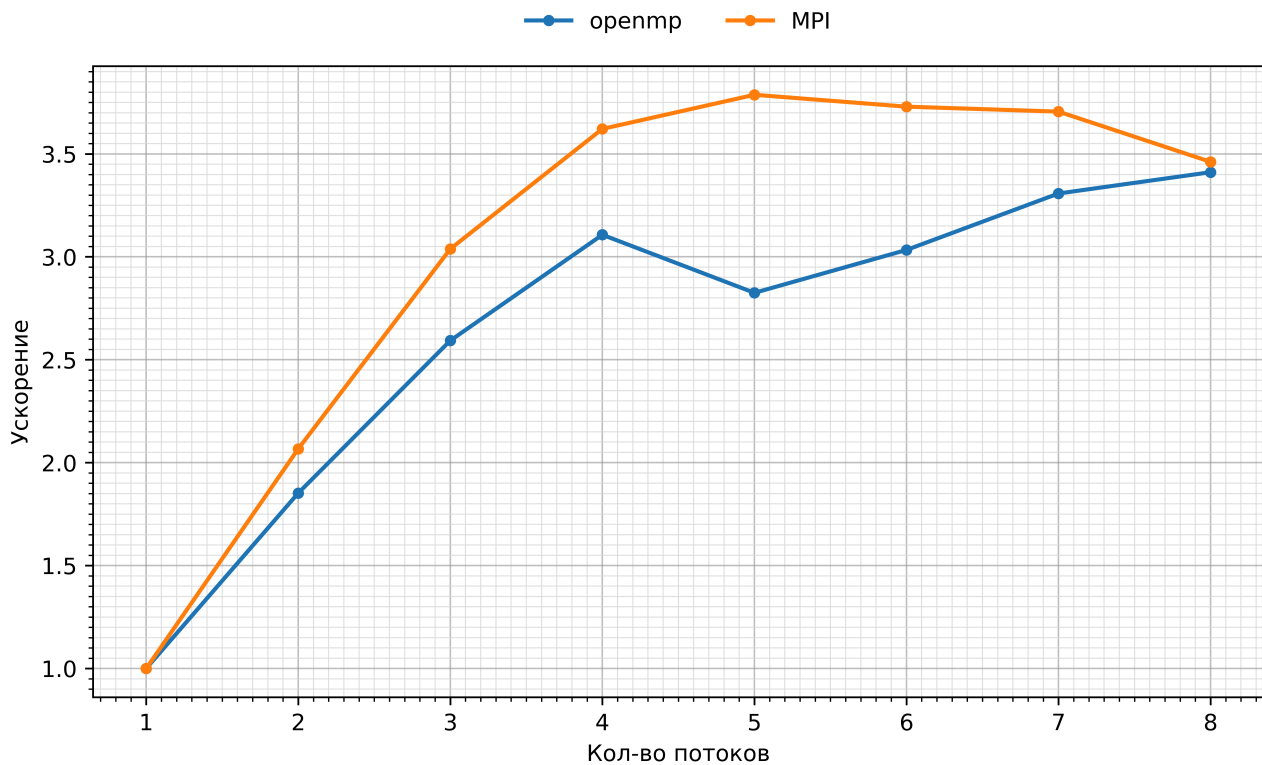


Рисунок 3 – График зависимости ускорения программы от количества потоков для MPI и OPENMP реализаций

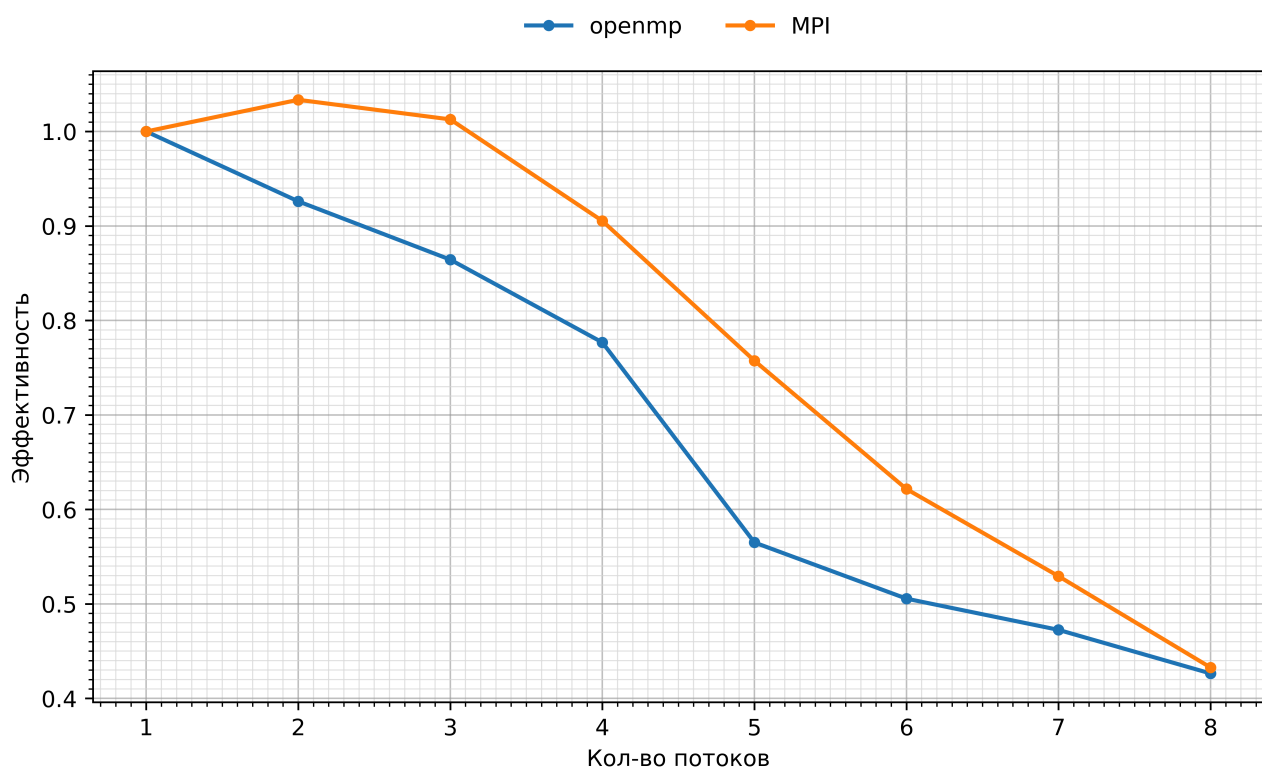


Рисунок 4 – График зависимости эффективности программы от количества потоков для MPI и OPENMP реализаций

4.1. Дополнительное задание

Сортировка Шелла начинается не с шага $\frac{n}{2}$, а с шага $\frac{n}{2^9}$.

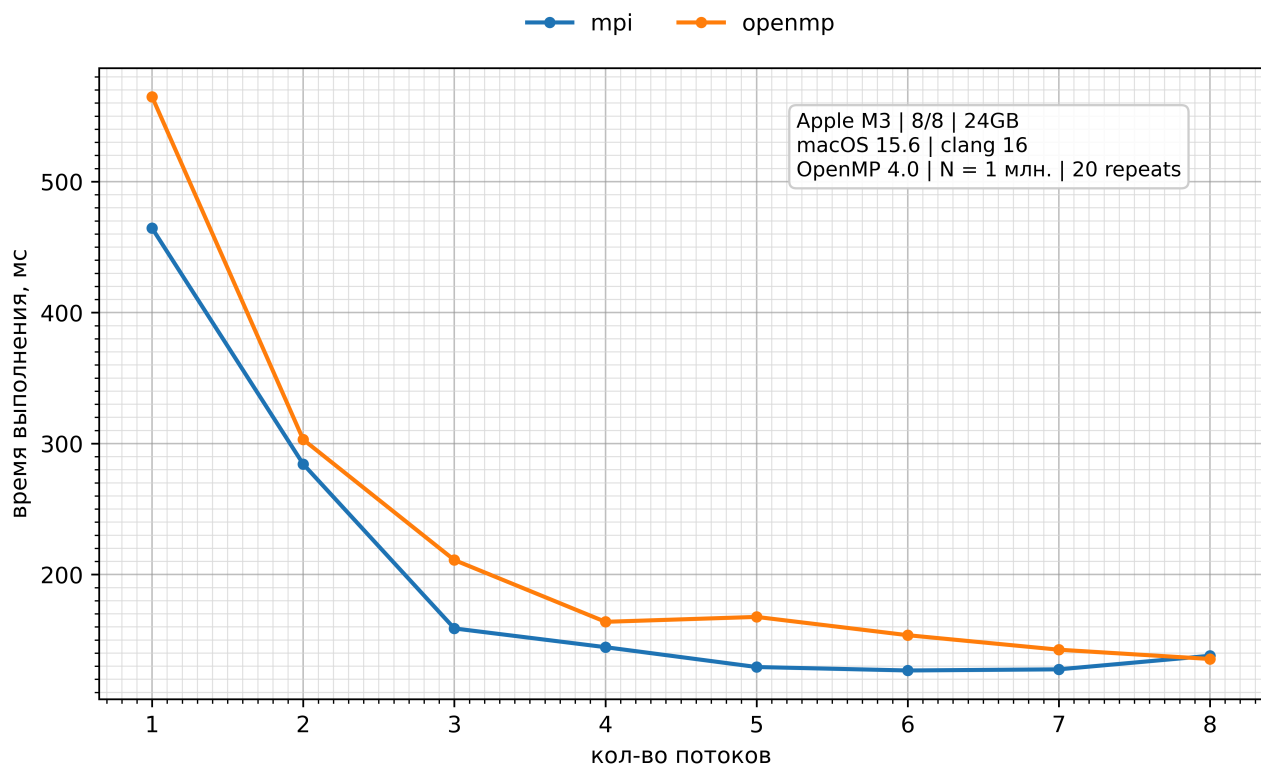


Рисунок 5 – График зависимости времени выполнения программы от количества потоков для MPI и OPENMP реализаций

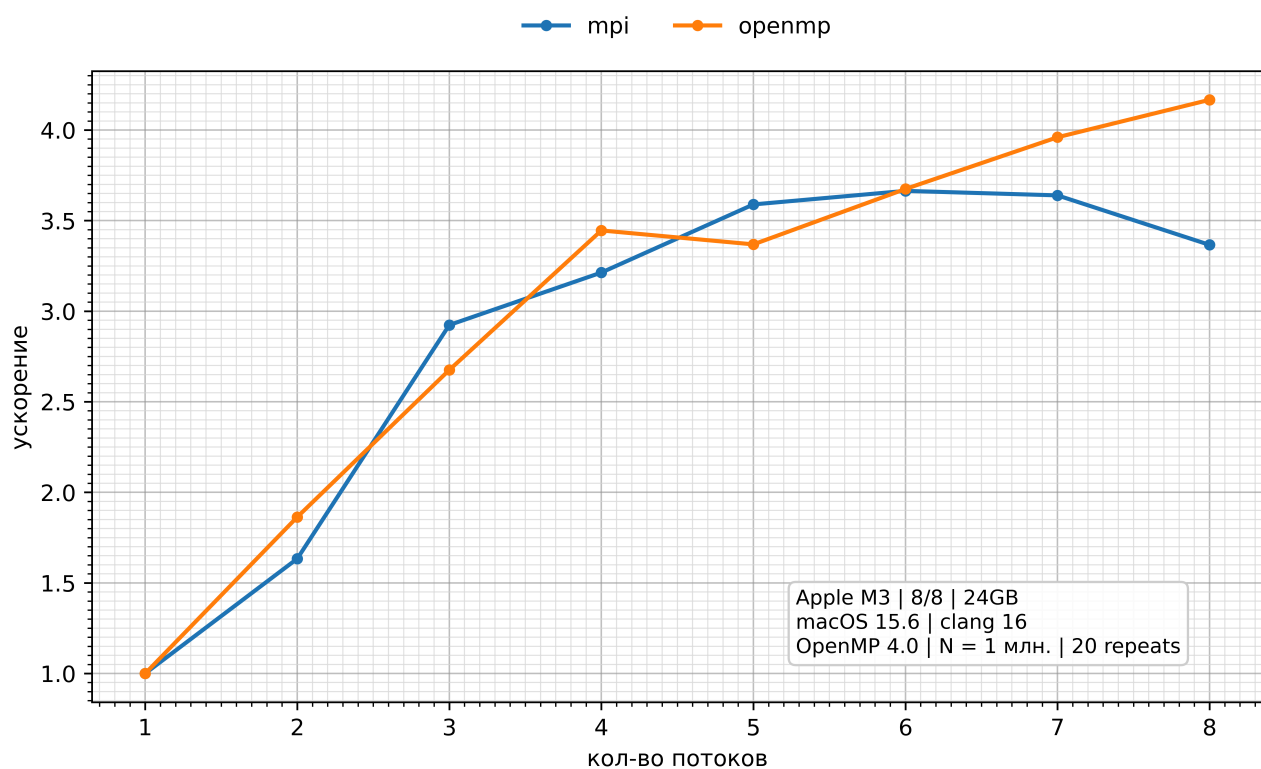


Рисунок 6 – График зависимости ускорения программы от количества потоков для MPI и OPENMP реализаций

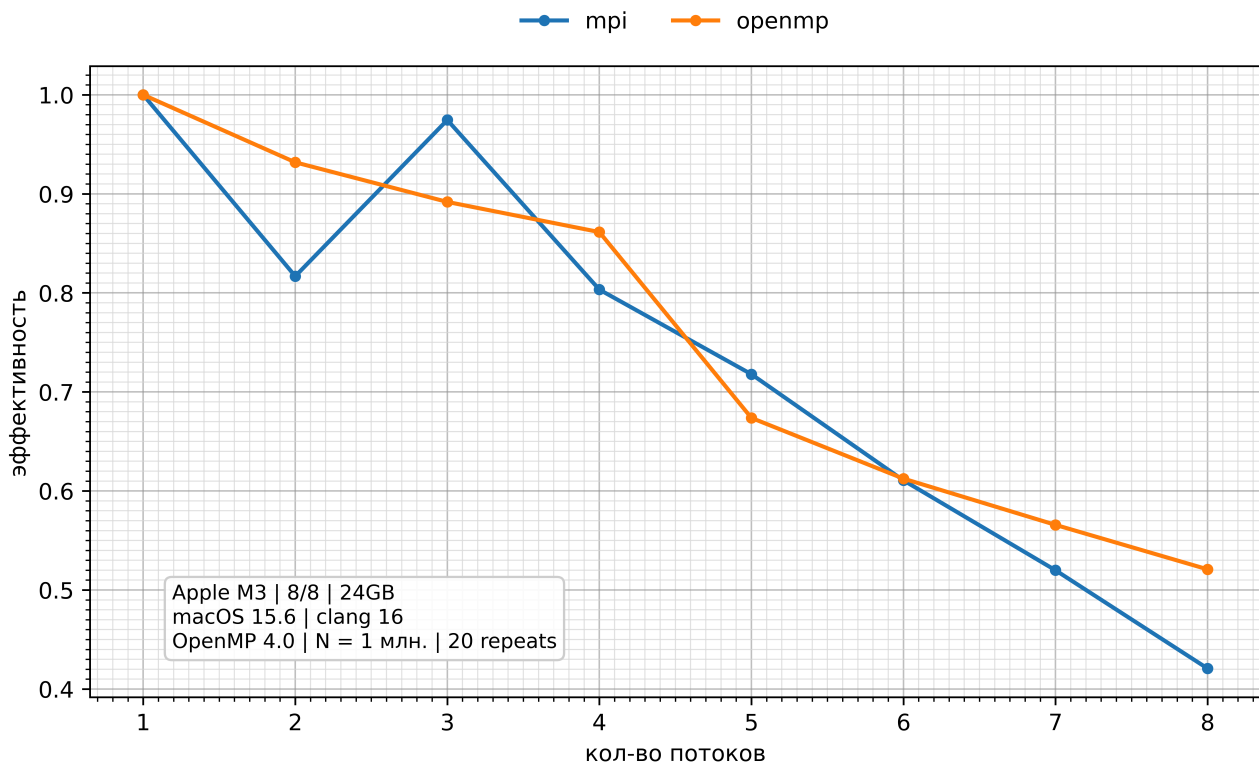


Рисунок 7 – График зависимости эффективности программы от количества потоков для MPI и OPENMP реализаций

5. Заключение

В рамках лабораторной работы была создана параллельная программа, реализующая алгоритм сортировки Шелла с использованием технологии OpenMPI. Также проведено сравнение с версией программы, использующей технологию OpenMP из лабораторной работы №3.

По результатам на графиках видно, что MPI обеспечивает меньшее время выполнения по сравнению с OpenMP на небольшом числе процессов/потоков (при 2–4 потоках/процессах MPI быстрее OpenMP). Однако по мере увеличения числа потоков преимущество MPI уменьшается и в OpenMP показывает сопоставимое время выполнения.

Для MPI наблюдается эффективность, превышающая единицу на 2–3 процессах. Это связано с улучшением локальности данных: при делении массива на несколько частей каждая часть лучше помещается в кэш процессора, что ускоряет локальную сортировку. В результате полное время работы программы на 2 процессах оказалось меньше половины времени работы на 1 процессе.

Эффективность OpenMP постепенно снижается с ростом числа потоков, что типично для задач с ограничением по кэшу и ресурсам памяти. В то же время эффективность MPI снижается медленнее на 2–4 процессах. При 5–8 процессах накладные расходы на MPI_Scatterv, MPI_Gatherv и слияние отсортированных блоков начинают играть значительную роль, значительно снижая эффективность.

На графиках наблюдается прекращение роста ускорения и эффективности после достижения 4 потоков. Это связано с устройством процессоров Apple серии M: они используют 4 производительных и 4 энергоэффективных ядра. При 4 потоках достигается предел количества высокопроизводительных ядер, что ограничивает дальнейший линейный рост производительности (подробное объяснение см. в заключении лабораторной №1).

Таким образом, для задач сортировки массивов в системах с общей памятью использование OpenMP предпочтительнее с точки зрения простоты реализации и предсказуемости масштабирования, тогда как MPI может быть оправдано для распределённых систем или при больших объёмах данных, превышающих локальную память одного узла.

Дополнительное задание. В сортировке Шелла шаги были изменены: сортировка начиналась с $\frac{n}{2^9}$ вместо стандартного $\frac{n}{2}$. На графике эффективности наблюдается резкое падение при 2 процессах и рост при 3. Это объясняется чувствительностью гар-последовательности: для разных размеров локальных массивов (`local_n`) при разном числе процессов формируются разные последовательности шагов, и некоторые наборы приводят к значительному увеличению числа сравнений и обменов.

Для отладки была разработана дополнительная функция сортировки Шелла, которая позволяет получить кол-во обменов и сравнений. Анализ показал: для 1 процесса `comps` = 181,038,727; для 2 процессов `comps` $\approx$ 110,024,300 на процесс (что суммарно на 21% больше, чем в 1-процессном случае); для 3 процессов суммарно `comps` $\approx$ 168,654,591 суммарно, что меньше, чем для 1 и 2 процессов. Это объясняет падение эффективности на 2 процессах.

6. Приложение. Разработанные программные коды

```
#include <mpi.h>
#include <stdint>
#include <vector>

struct GapInfo {
    int gap;
    uint64_t comps;
    uint64_t swaps;
    double time_ms;
};

struct SortStats {
    double total_ms = 0.0;
    uint64_t comps = 0;
    uint64_t swaps = 0;
    std::vector<GapInfo> gaps;
};

SortStats shell_sort_instrumented(int *array, int size) {
    SortStats st;
    double t0 = MPI_Wtime();
    for (int s = size / 512; s > 0; s /= 2) {
        GapInfo gi;
        gi.gap = s;
        gi.comps = 0;
        gi.swaps = 0;
        double g0 = MPI_Wtime();

        for (int offset = 0; offset < s; ++offset) {
            for (int i = offset + s; i < size; i += s) {
                for (int j = i - s; j >= 0; j -= s) {
                    ++gi.comps;
                    if (array[j] > array[j + s]) {
                        ++gi.swaps;
                        int tmp = array[j];
                        array[j] = array[j + s];
                        array[j + s] = tmp;
                    } else {
                        break;
                    }
                }
            }
        }

        double g1 = MPI_Wtime();
        gi.time_ms = (g1 - g0) * 1000.0;
        st.gaps.push_back(gi);
        st.comps += gi.comps;
        st.swaps += gi.swaps;
    }
    double t1 = MPI_Wtime();
    st.total_ms = (t1 - t0) * 1000.0;
    return st;
}
```

```
#include "shell.h"
#include <mpi.h>
#include <iostream>
#include <fstream>
#include <memory>
#include <random>
#include <vector>
#include <climits>
#include <cmath>
#include <iomanip>
#include <ctime>

namespace fs = std::filesystem;

static const int num_arrays = 20;
static const int size = std::pow(10, 6);

std::random_device dev;
std::mt19937 gen(dev());
std::uniform_int_distribution<int> dist(0, 1000000);

std::unique_ptr<int[]> gen_random_array(int size) {
    std::unique_ptr<int[]> array = std::make_unique<int[]>(size);
    for (int i = 0; i < size; ++i)
        array[i] = dist(gen);
    return array;
}

int main(int argc, char** argv) {
    int rank, processes;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &processes);

    double total_time_mpi = 0.0;

    std::vector<std::unique_ptr<int[]>> all_arrays;
    all_arrays.reserve(num_arrays);
    for (int i = 0; i < num_arrays; ++i)
        all_arrays.push_back(gen_random_array(size));

    std::unique_ptr<int[]> sizes_arr = std::make_unique<int[]>(processes);
    std::unique_ptr<int[]> offsets_arr = std::make_unique<int[]>(processes);
    int base_chunk = size / processes;
    int rem = size % processes;
    for (int i = 0; i < processes; i++) {
        sizes_arr[i] = base_chunk + (i < rem ? 1 : 0);
        offsets_arr[i] = (i == 0) ? 0 : offsets_arr[i - 1] + sizes_arr[i - 1];
    }

    int local_n = sizes_arr[rank];
    std::unique_ptr<int[]> local_arr = std::make_unique<int[]>(local_n);

    std::unique_ptr<int[]> mpi_merge_buf = nullptr;
```

```

if (rank == 0)
    mpi_merge_buf = std::make_unique<int[]>(size);

for (int array_index = 0; array_index < num_arrays; ++array_index) {
    std::unique_ptr<int[]> global_arr = nullptr;
    if (rank == 0) {
        global_arr = std::make_unique<int[]>(size);
        memcpy(global_arr.get(), all_arrays[array_index].get(), size * sizeof(int));
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double start = MPI_Wtime();

    MPI_Scatterv(
        global_arr.get(), sizes_arr.get(), offsets_arr.get(), MPI_INT,
        local_arr.get(), local_n, MPI_INT,
        0, MPI_COMM_WORLD
    );
    shell_sort(local_arr.get(), local_n);

    MPI_Gatherv(
        local_arr.get(), local_n, MPI_INT,
        mpi_merge_buf.get(), sizes_arr.get(), offsets_arr.get(),
        MPI_INT, 0, MPI_COMM_WORLD
    );

    if (rank == 0) {
        std::vector<int*> subarrays(processes);
        for (int p = 0; p < processes; ++p)
            subarrays[p] = mpi_merge_buf.get() + offsets_arr[p];
        merge_arrays(mpi_merge_buf.get(), subarrays.data(), processes, sizes_arr.get(), size);
    }

    MPI_Barrier(MPI_COMM_WORLD);

    if (rank == 0) {
        double end = MPI_Wtime();
        total_time_mpi += (end - start);
        for (int i=0; i<size-1; ++i) {
            if (mpi_merge_buf[i] > mpi_merge_buf[i+1]) {
                std::cout << "NOT SORTED" << std::endl;
            }
        }
    }
}

if (rank == 0) {
    double avg_time_mpi = total_time_mpi / num_arrays;
    double avg_mpi_ms = avg_time_mpi * 1000.0;

    double time_one_mpi = 0.0;
    std::string baseline_fname = "results/baseline_512";

    if (processes == 1) {
        std::ofstream base(baseline_fname);
        if (base) {

```

```

        base << std::fixed << std::setprecision(6) << avg_mpi_ms << "\n";
        time_one_mpi = avg_mpi_ms;
    } else {
        std::cerr << "Cannot write baseline file " << baseline_fname << "\n";
    }
} else {
    if (fs::exists(baseline_fname) && fs::file_size(baseline_fname) > 0) {
        std::ifstream base(baseline_fname);
        if (base) {
            base >> time_one_mpi;
        }
    }
}

double real_speedup_mpi = time_one_mpi / avg_mpi_ms;

double efficiency_mpi = real_speedup_mpi / processes;

std::string fname_time = "results/time_512";
bool need_header_time = (!fs::exists(fname_time) || fs::file_size(fname_time) == 0);
{
    std::ofstream f(fname_time, std::ios::app);
    if (need_header_time) {
        f << 1 << "\n";
        f << "MPI\n";
        f << "Размер массива\n";
        f << "Время выполнения, мс\n";
        f << "none\n";
    }
    f << processes << " " << avg_mpi_ms << "\n";
}

std::string fname_sp = "results/speedup_512";
bool need_header_sp = (!fs::exists(fname_sp) || fs::file_size(fname_sp) == 0);
{
    std::ofstream f(fname_sp, std::ios::app);
    if (need_header_sp) {
        f << 1 << "\n";
        f << "MPI\n";
        f << "Размер массива\n";
        f << "Ускорение\n";
        f << "none\n";
    }
    f << processes << " " << real_speedup_mpi << "\n";
}

std::string fname_ef = "results/efficiency_512";
bool need_header_ef = (!fs::exists(fname_ef) || fs::file_size(fname_ef) == 0);
{
    std::ofstream f(fname_ef, std::ios::app);
    if (need_header_ef) {
        f << 1 << "\n";
        f << "MPI\n";
        f << "Размер массива\n";
        f << "Эффективность\n";
        f << "none\n";
    }
}

```

```

        f << processes << " " << efficiency_mpi << "\n";
    }

    std::cout << "Finished. Processes=" << processes
               << " MPI_avg_time=" << avg_mpi_ms << " ms\n";
}

MPI_Finalize();
return 0;
}

```

Файл 2 – Измерение времени для параллельной реализации